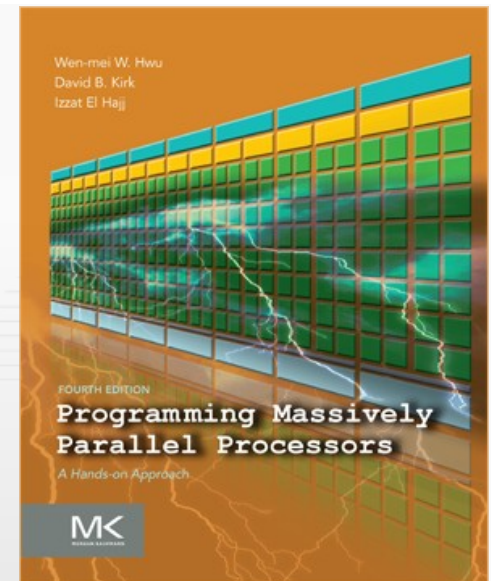


Programming Massively Parallel Processors

A Hands-on Approach

CHAPTER 10 > Reduction



- A **reduction** operation reduces a set of input values to one value
 - e.g., sum, product, min, max
- Reduction operations are:
 - Associative
 - Commutative
 - Have a well-define identity value
- We will use sum as an example

- Sequential reduction for sum:

```
sum = 0.0f;  
for(i = 0; i < N; ++i) {  
    sum += input[i];  
}
```

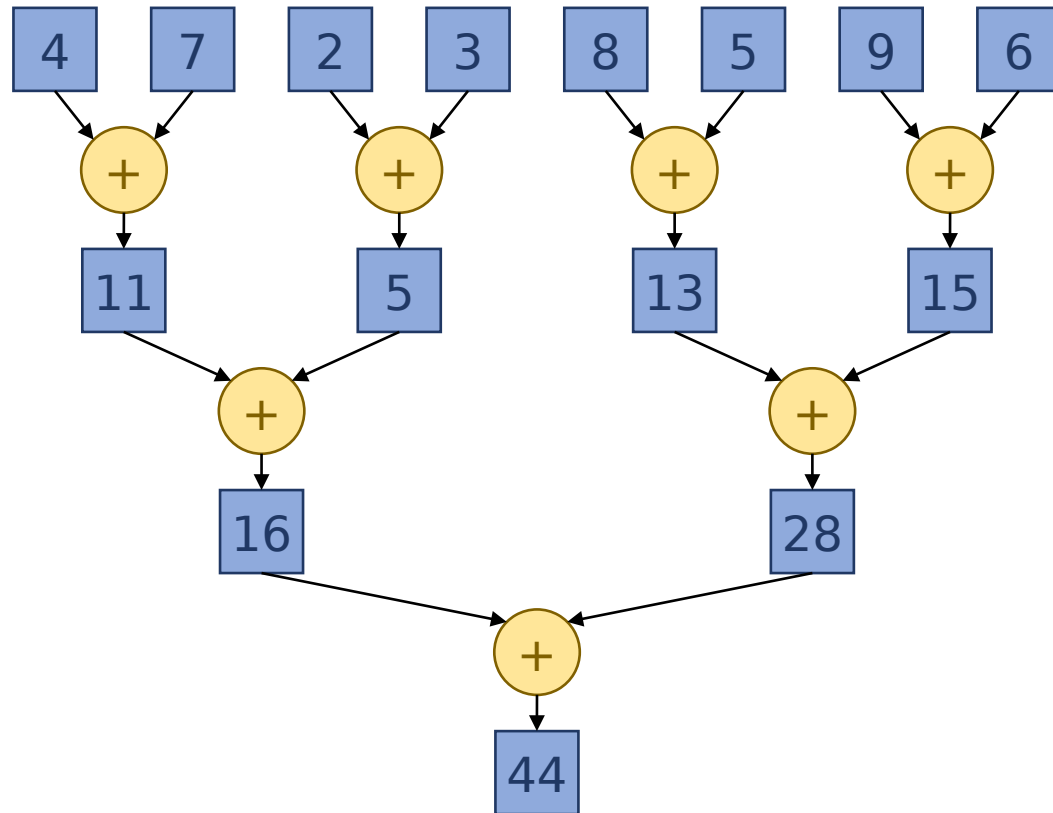
- In general:

```
acc = IDENTITY;  
for(i = 0; i < N; ++i) {  
    acc = f(acc, input[i]);  
}
```

- Parallel reduction for sum using atomics:

```
unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;  
if(i < N) {  
    atomicAdd(sum, input[i]);  
}
```

- Poor performance:
 - All additions are serialized by the hardware



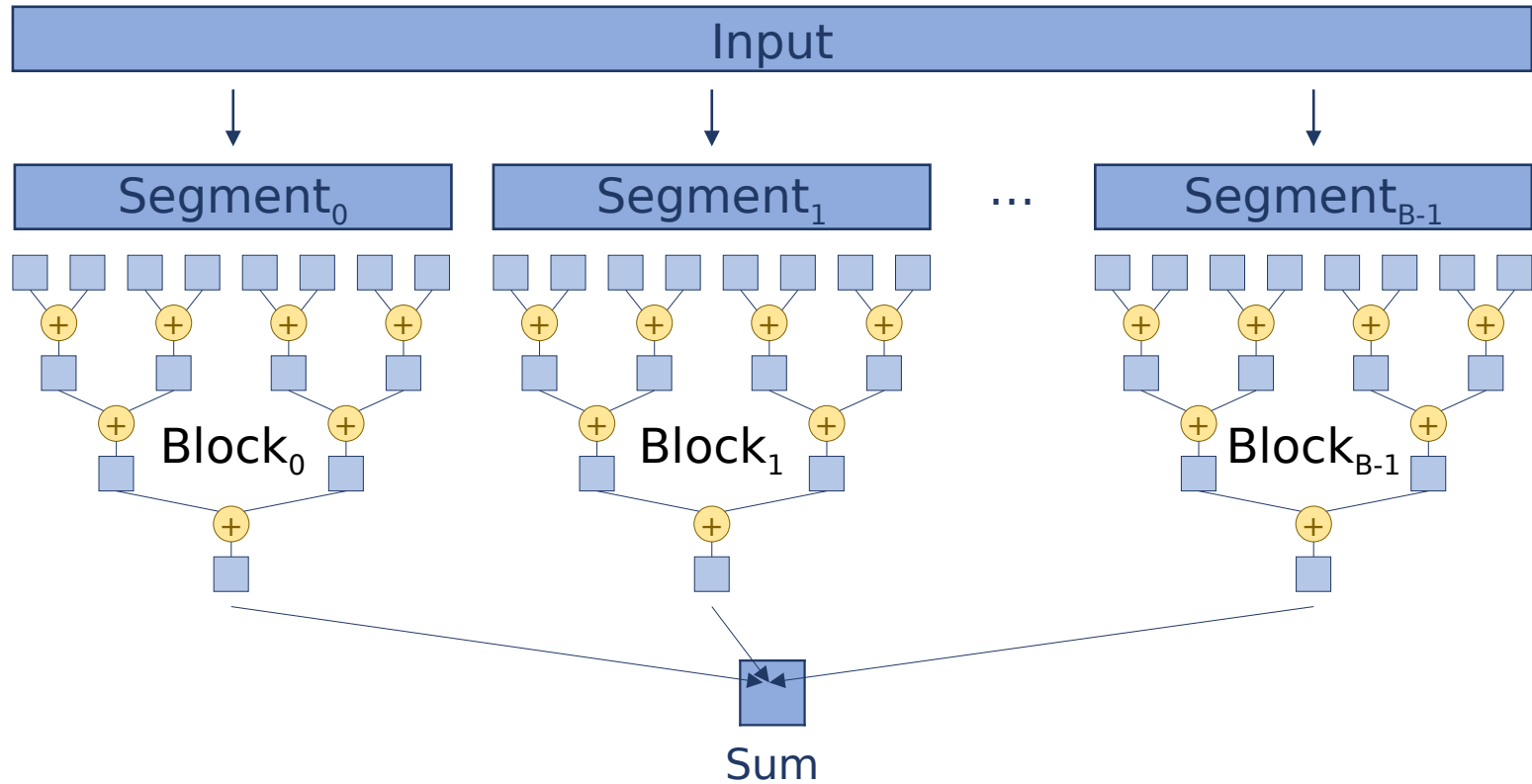
Approach: Every thread adds two elements in each step

Takes $\log(N)$ steps and half the threads drop out every step

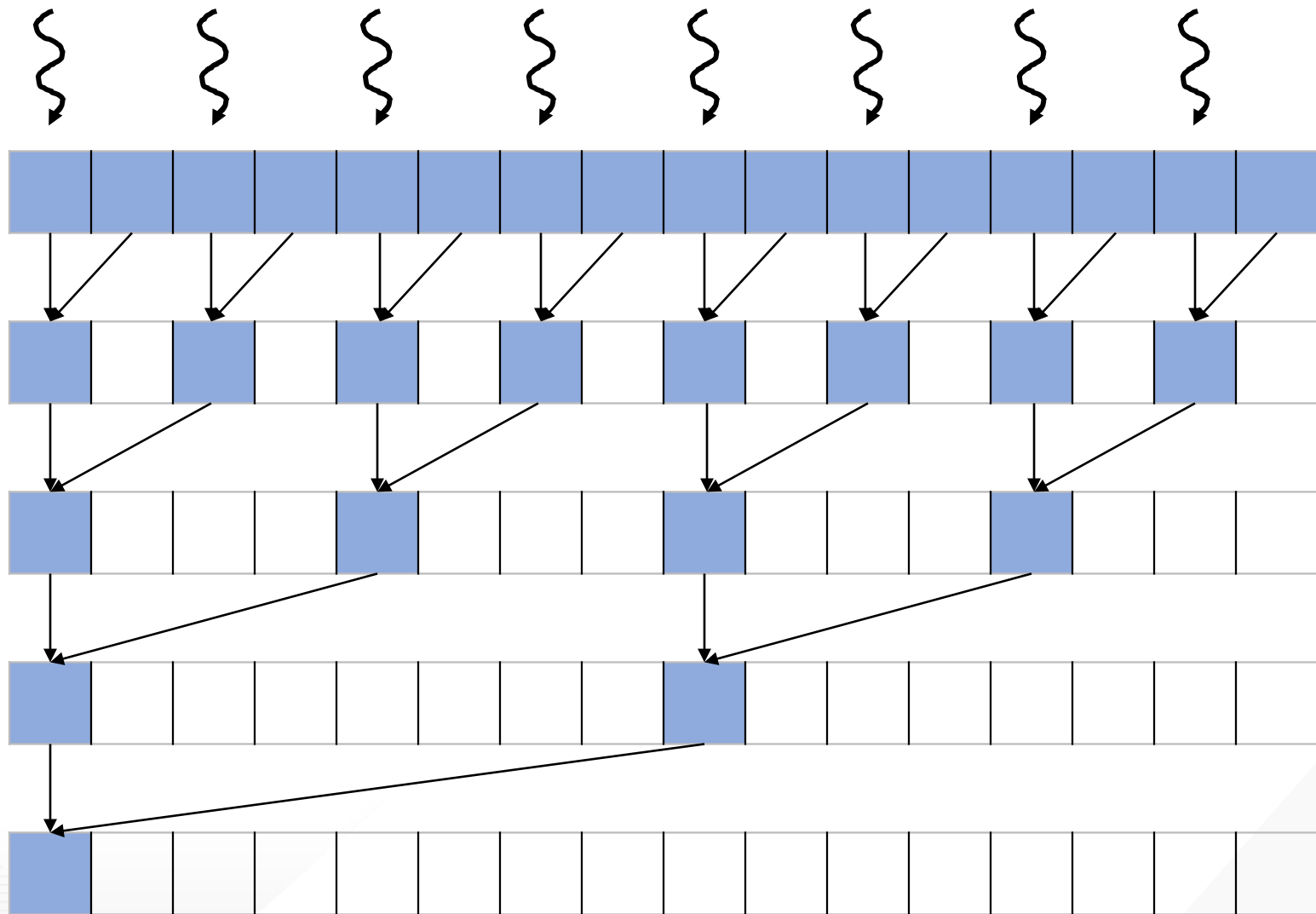
Pattern is called a **reduction tree**

- Threads must synchronize between steps
 - Cannot synchronize across threads in different blocks
- Solution: **segmented reduction**
 - Every thread block reduces a segment of the input and produces a partial sum
 - The partial sum is atomically added to the final sum

Segmented Reduction Example

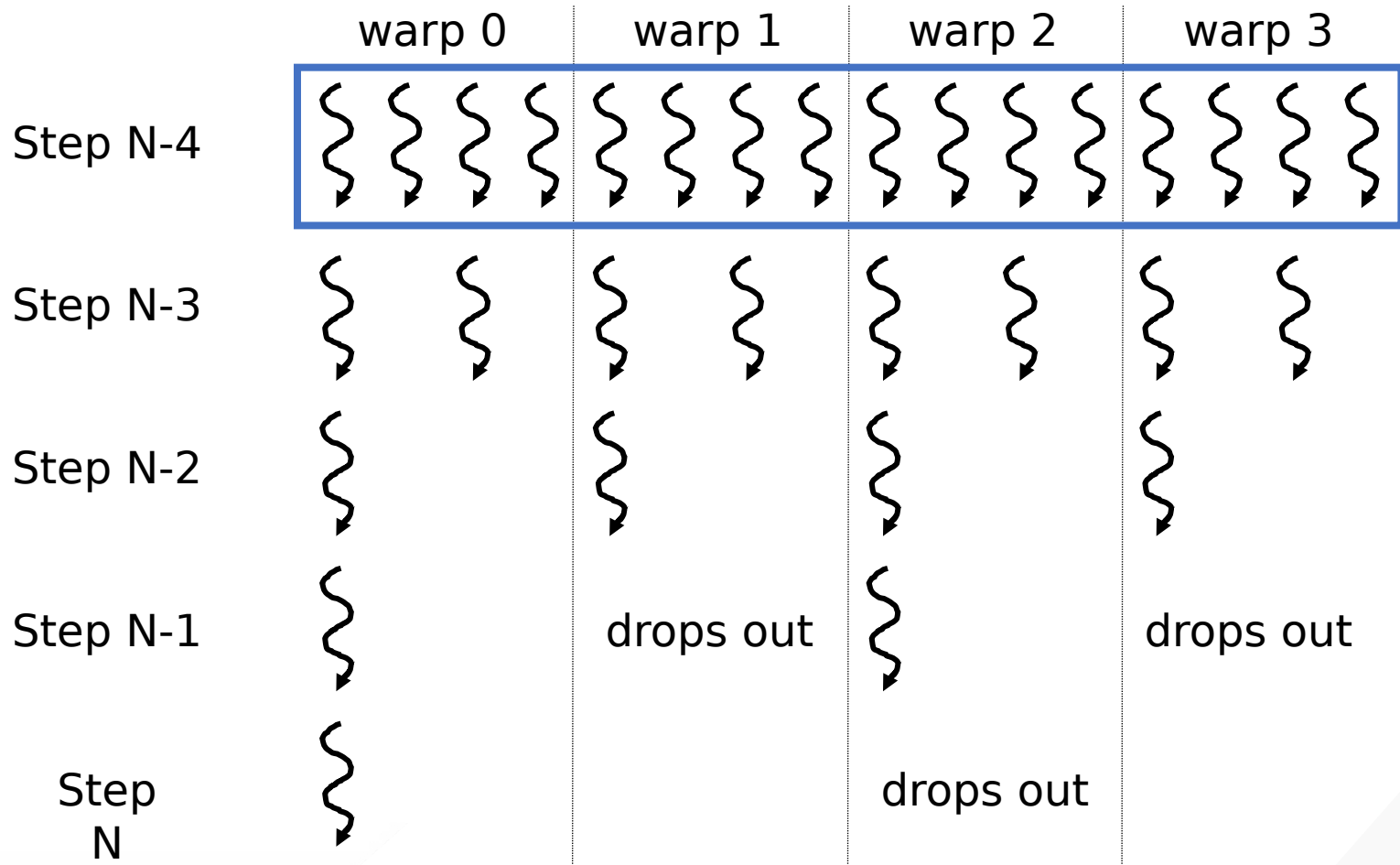


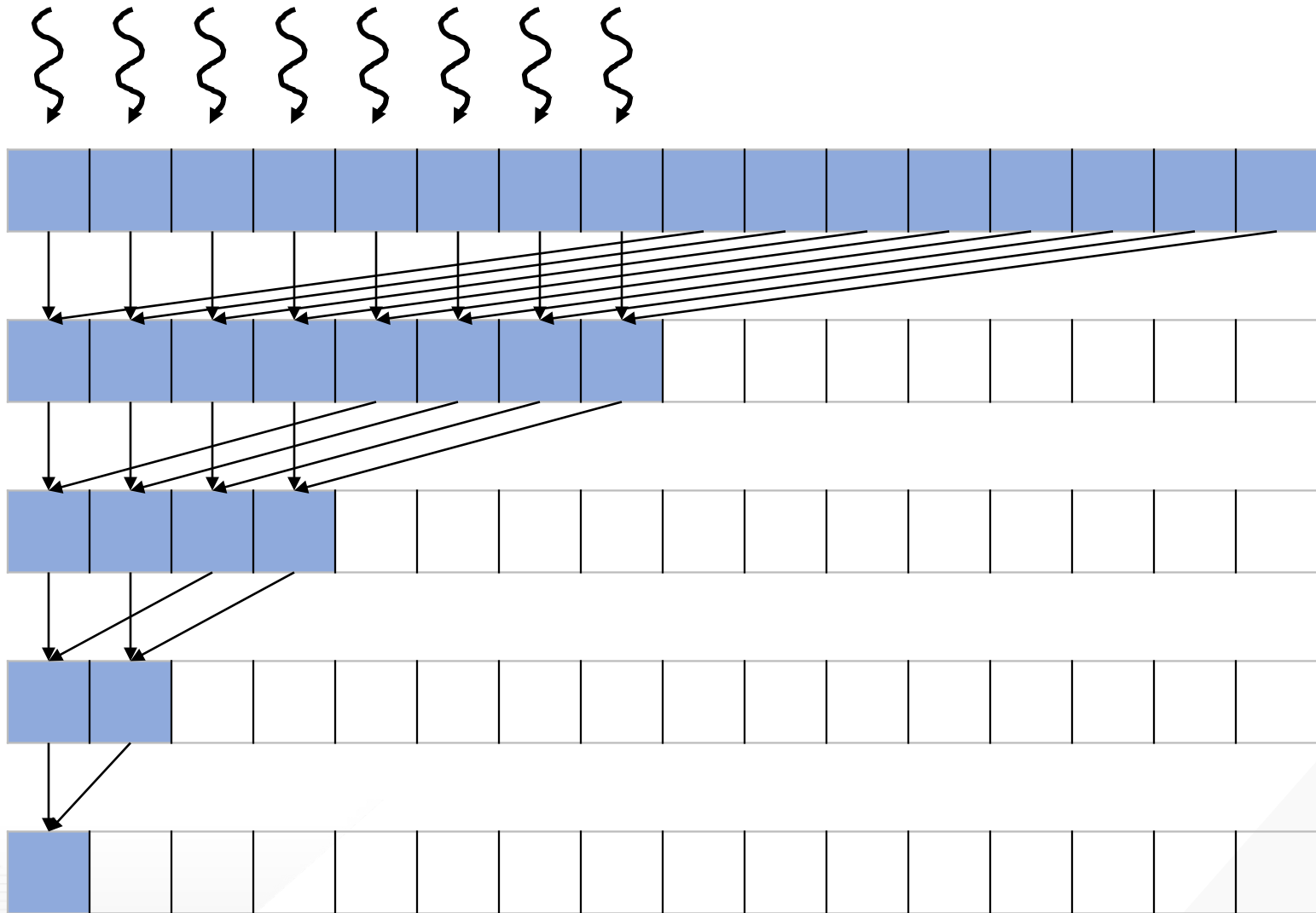
Reduction Tree (Per Block)

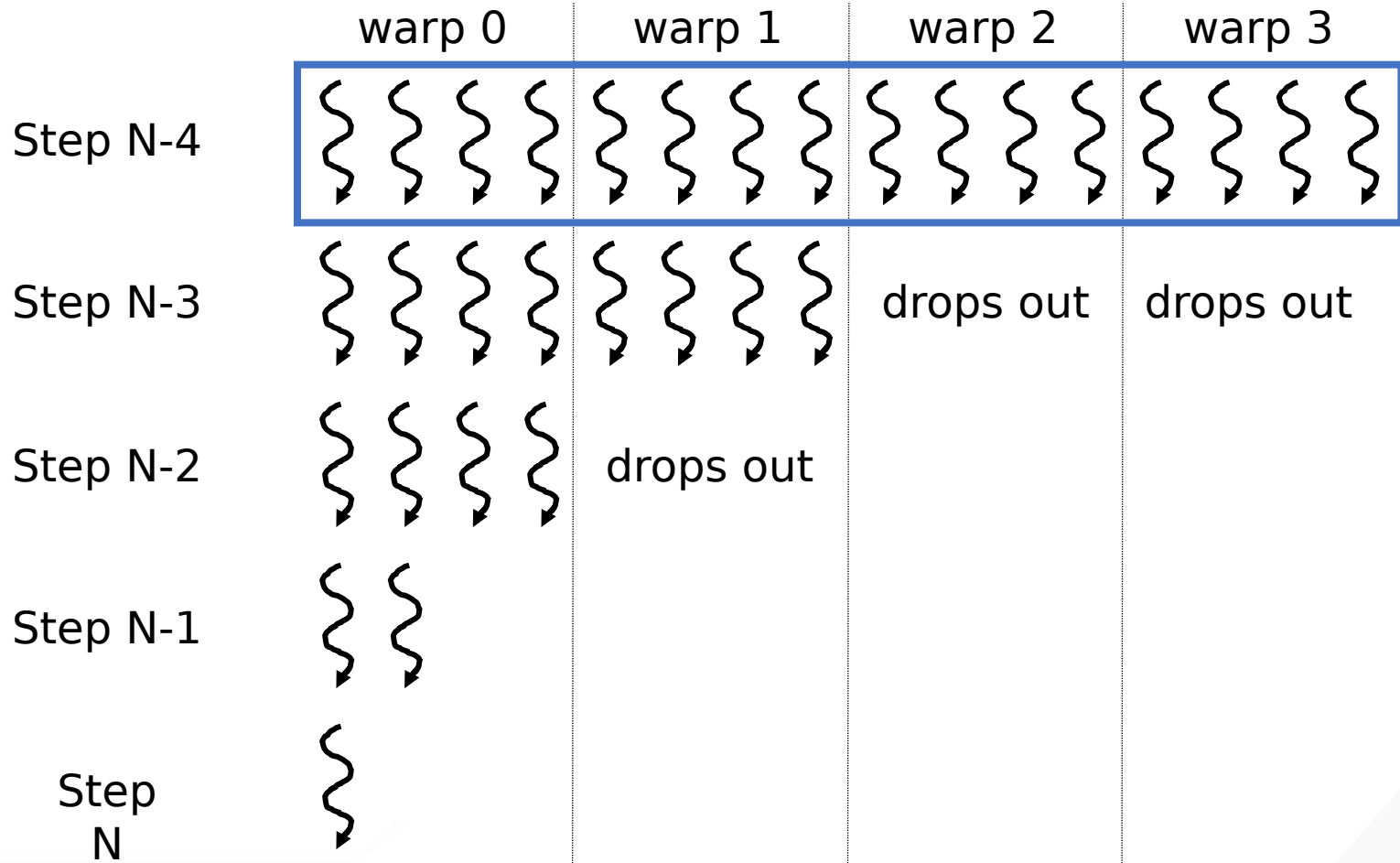



```
unsigned int segment = 2*blockDim.x*blockIdx.x;
unsigned int i = segment + 2*threadIdx.x;
for(unsigned int stride = 1; stride <= BLOCK_DIM; stride *= 2) {
    if(threadIdx.x%stride == 0) {
        input[i] += input[i + stride];
    }
    __syncthreads();
}
```

- Problems:
 - Accesses to input are not coalesced
 - Control divergence

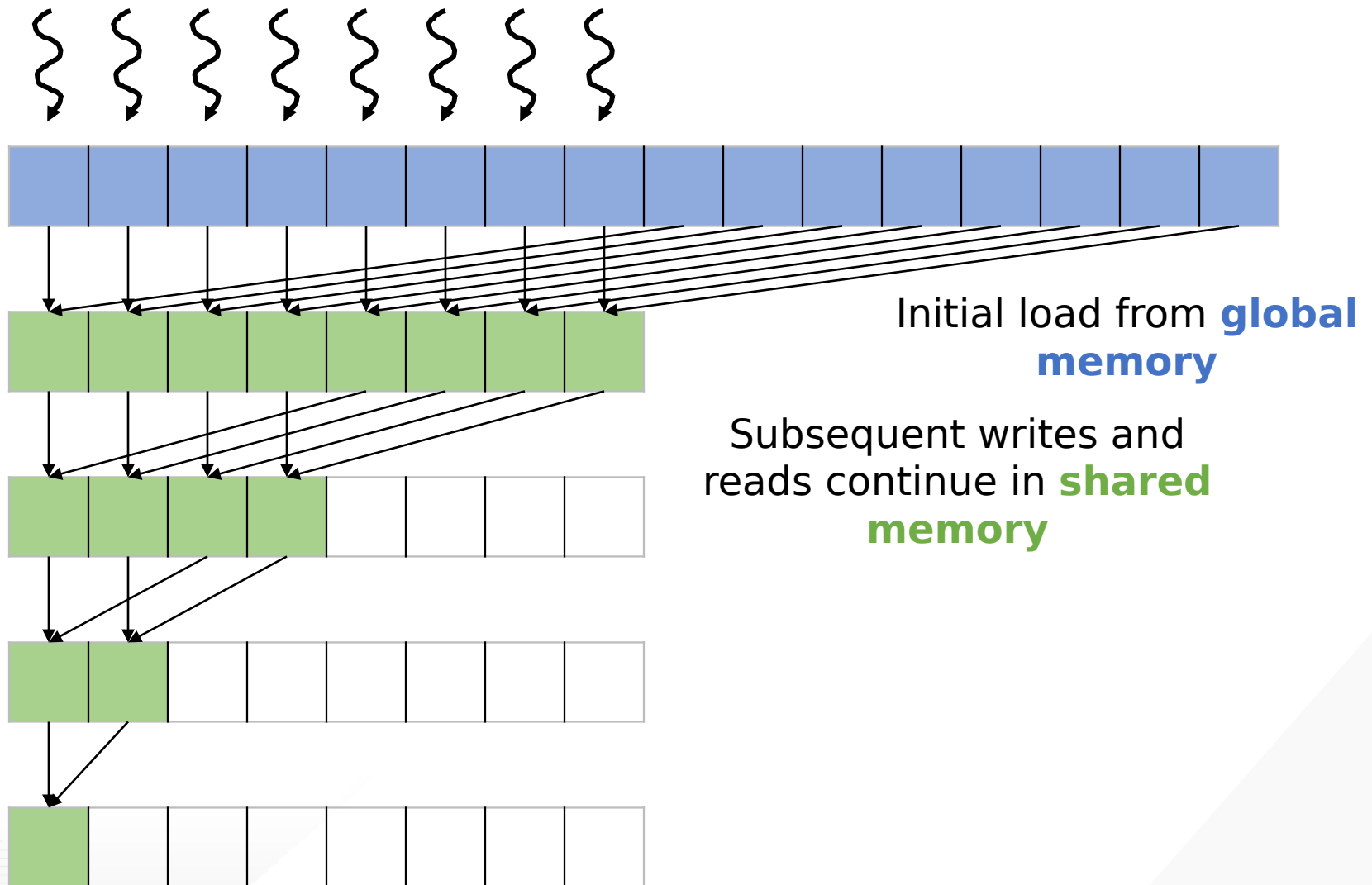






```
unsigned int segment = 2*blockDim.x*blockIdx.x;
unsigned int i = segment + threadIdx.x;
for(unsigned int stride = BLOCK_DIM; stride > 0; stride /= 2) {
    if(threadIdx.x < stride) {
        input[i] += input[i + stride];
    }
    __syncthreads();
}
```

- While specific data values are not reused, the same memory locations are repeatedly read and written
- Optimization: load input to shared memory first and perform reduction tree on shared memory
 - Also avoids modifying the input if needed in the future

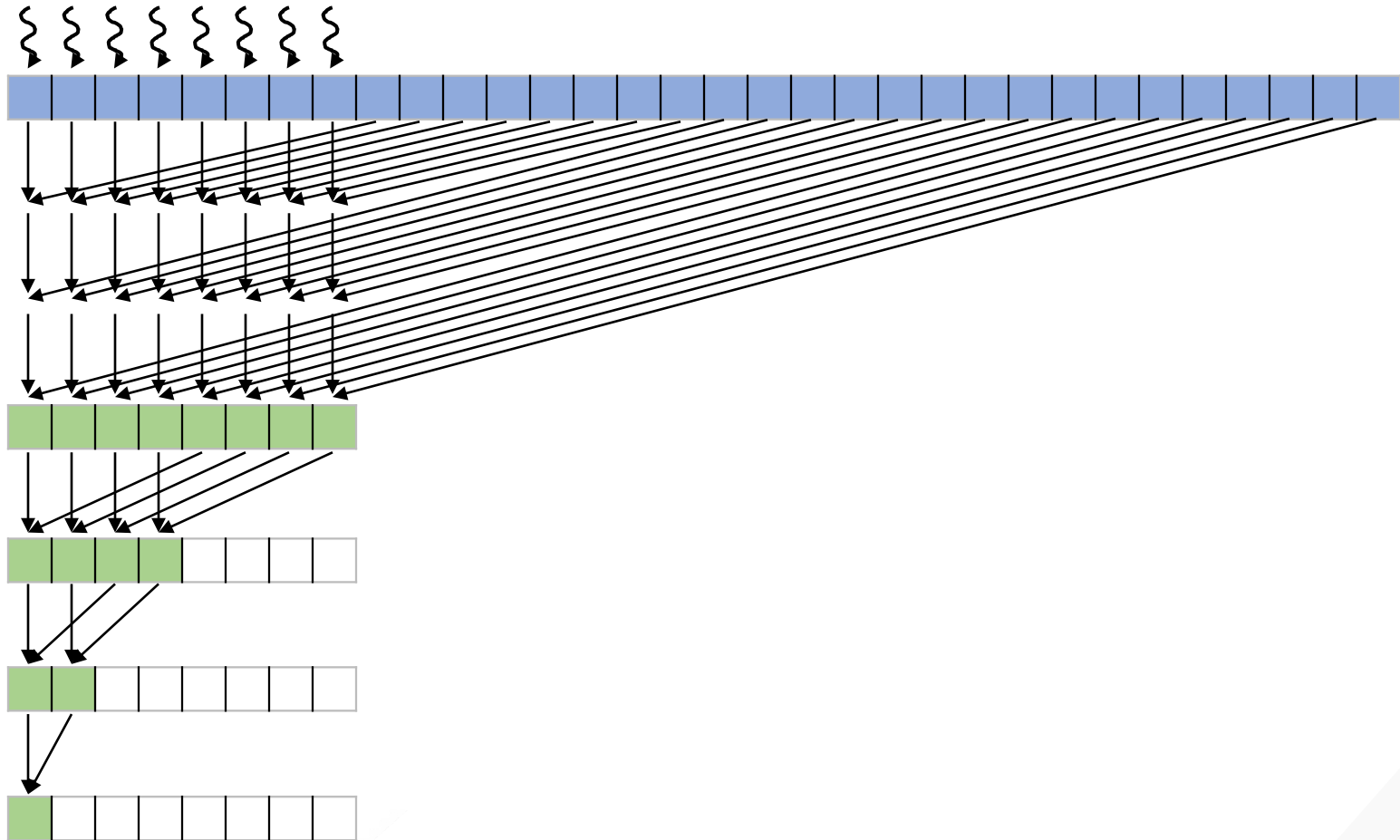


```
unsigned int segment = 2*blockDim.x*blockIdx.x;
unsigned int i = segment + threadIdx.x;

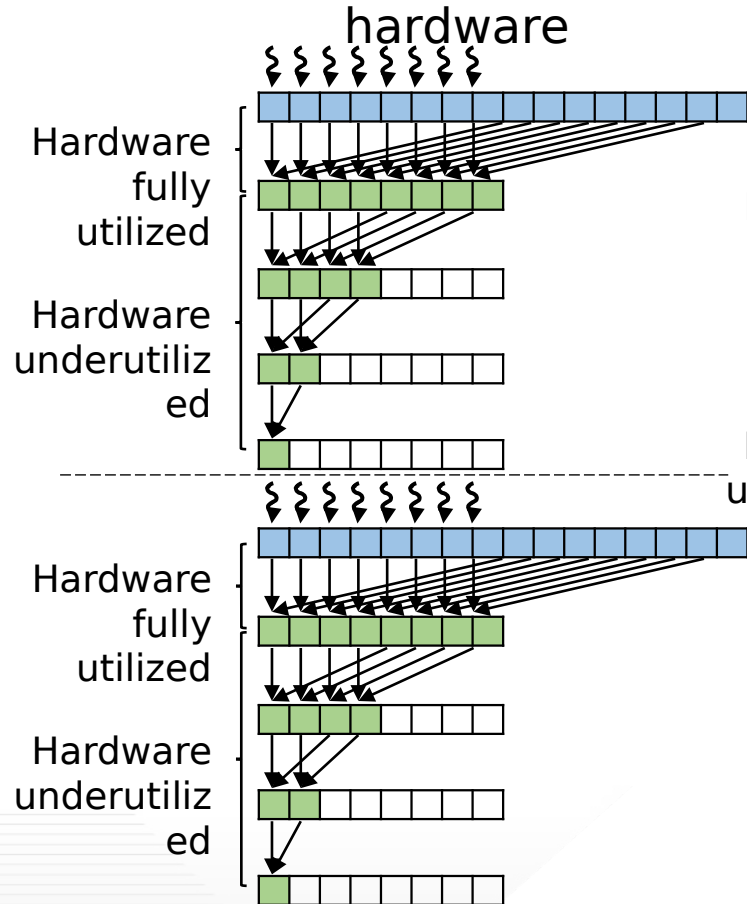
// Load data to shared memory
__shared__ float input_s[BLOCK_DIM];
input_s[threadIdx.x] = input[i] + input[i + BLOCK_DIM];
__syncthreads();

// Reduction tree in shared memory
for(unsigned int stride = BLOCK_DIM/2; stride > 0; stride /= 2)
{
    if(threadIdx.x < stride) {
        input_s[threadIdx.x] += input_s[threadIdx.x + stride];
    }
    __syncthreads();
}
```

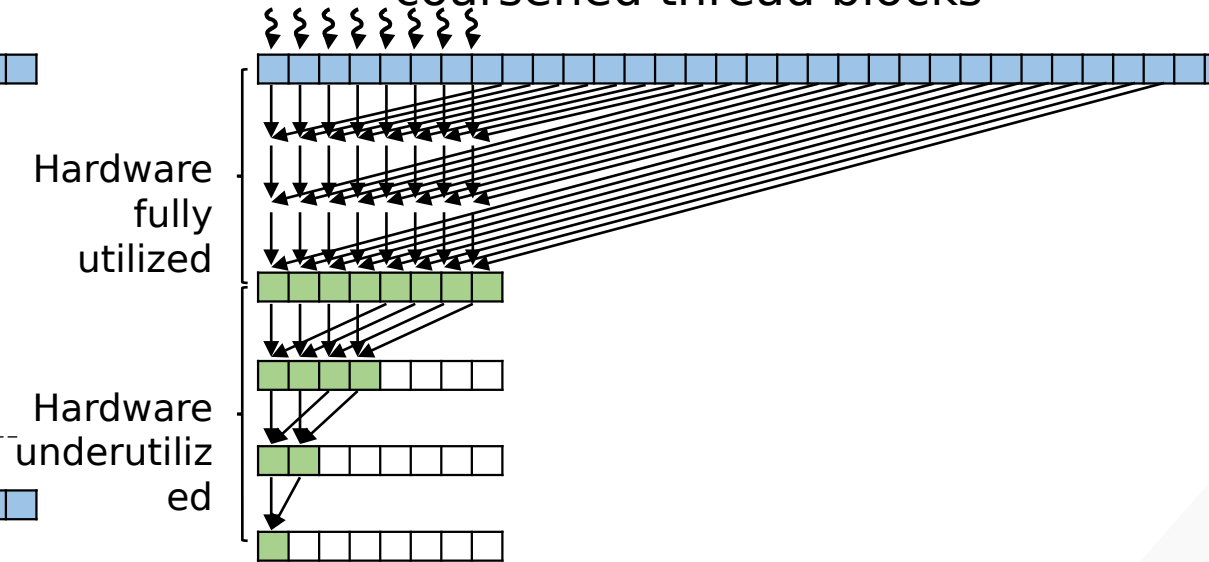

- Cost of parallelization:
 - Synchronization every step
 - Control divergence in the final steps
- Better to coarsen threads if there are many more blocks than resources available



Execution of two non-coarsened thread blocks serialized by the hardware



Execution of one coarsened thread block doing the work of two original non-coarsened thread blocks



```

unsigned int segment = COARSE_FACTOR*2*blockDim.x*blockIdx.x;
unsigned int i = segment + threadIdx.x;

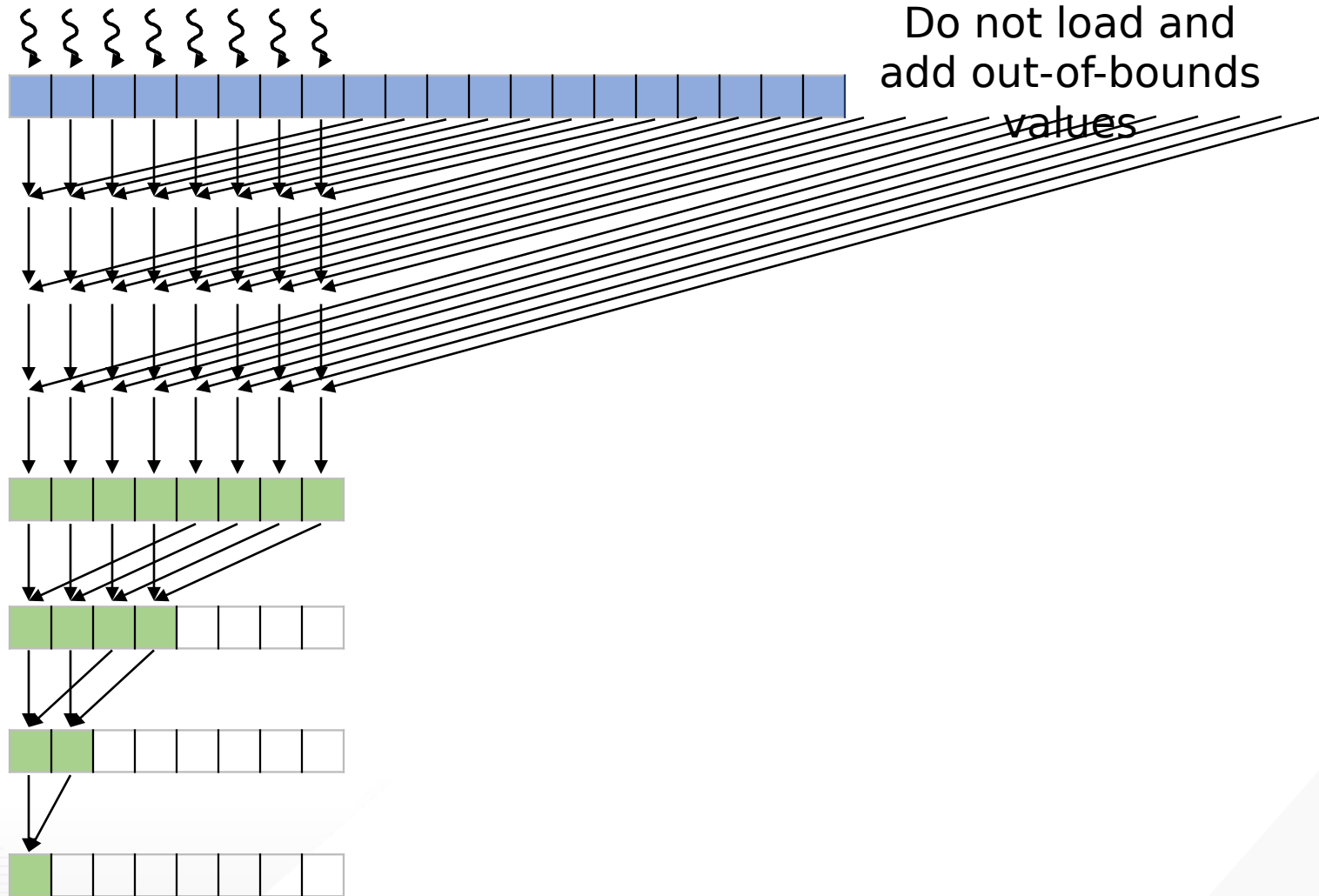
// Load data to shared memory
__shared__ float input_s[BLOCK_DIM];
float threadSum = 0.0f;
for(unsigned int c = 0; c < COARSE_FACTOR*2; ++c) {
    threadSum += input[i + c*BLOCK_DIM];
}
input_s[threadIdx.x] = threadSum;

__syncthreads();

// Reduction tree in shared memory
for(unsigned int stride = BLOCK_DIM/2; stride > 0; stride /= 2)
{
    if(threadIdx.x < stride) {
        input_s[threadIdx.x] += input_s[threadIdx.x + stride];
    }
    __syncthreads();
}

```

- Let N be the number of elements per original block
 - i.e., $N = 2 * \text{blockDim.x}$
- If blocks are all executed in parallel:
 - $\log(N)$ steps, $\log(N)$ synchronizations
- If blocks serialized by the hardware by a factor of C :
 - $C * \log(N)$ steps, $C * \log(N)$ synchronizations
- If blocks are coarsened by a factor of C :
 - $2 * (C - 1) + \log(N)$ steps, $\log(N)$ synchronizations



- Wen-mei W. Hwu, David B. Kirk, and Izzat El Hajj. *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 2022.